



Java-API Grundlagen

Brief Overview

This note covers **Java-API Grundlagen** and was created from a [34-Seiten PDF](#). Die Notizen bieten einen kompakten Überblick über Wrapper-Klassen, JAR-Dateien, I/O-Streams und praktische Codebeispiele.

Key Points

- **Wrapper-Klassen** für primitive Typen und generische Strukturen
- **JAR-Dateien** erstellen, ausführen und Inhalte anzeigen
- I/O-Streams: Input-/Output-Streams, Byte- und Character-Streams
- Beispiel-Code zum Datei-Einlesen/Schreiben mit FileReader/FileWriter



Nützliches aus der Java-API

Die **Java-API** (Application Programming Interface) stellt mit jeder Version erweiterte Klassen und Hilfsmittel bereit, die die Entwicklung erleichtern.



Wrapper-Klassen

Wrapper-Klassen kapseln primitive Datentypen in Objekten, sodass sie als Referenzen verwendet werden können.

Wrapper-Klasse	Primitiver Datentyp
Byte	byte
Short	short
Integer	int
Long	long
Float	float
Double	double
Boolean	boolean
Character	char

- **Aufgaben der Wrapper-Klassen**
 1. Statische Hilfsmethoden zur **Konvertierung** eines primitiven Werts in einen **String** (Formatierung).
 2. Statische Methoden zum **Parsen** eines Strings zurück in den entsprechenden primitiven Typ.
 3. Ermöglichen die Nutzung generischer Datenstrukturen (z.B. List), weil nur **Objekt-Referenzen** gespeichert werden können.



JAR-Dateien

- **Ausführen**: java -jar .jar
- **Inhalt anzeigen**: jar -tvf .jar
- **Extrahieren**: jar -xvf .jar

In Eclipse können JAR-Dateien direkt erzeugt werden (Details im Unterrichtsmaterial).

I/O Grundlagen – Streams

Ein **Stream** (dt. Strom) ist eine FIFO-Datenstruktur, die Informationen von einer Quelle zu einem Ziel transportiert.

- **Arten von Streams**
 - **Input-Stream**: transportiert Daten *von* einer Quelle *zu* einem Java-Programm.
 - **Output-Stream**: transportiert Daten *vom* Java-Programm *zu* einer Senke (Datei, Gerät, Netzwerk).
- **Paket**: Alle Stream-Klassen befinden sich im Paket java.io.
- **Vorteil**: Java-Entwickler arbeiten nicht direkt mit betriebssystemabhängigen Details; das OS übernimmt das Lesen/Schreiben von externen Quellen und Senken.

Quellen und Senken

Quelle / Senke	Intern (im Speicher)	Extern (außerhalb)
Interne Quellen	byte[], char[], String, Pipes	—
Externe Quellen	—	Dateien, Geräte, Netzverbindungen

- **Unterschiede**
 - **Datenart** (Byte- vs. Character-Streams)
 - **End-Verhalten** (z. B. feste Dateilänge vs. unbestimmtes Tastatureingabe-Ende)

Byte- und Character-Streams

- **Binärdaten** (z. B. Bilder, .exe, .class) → **Byte-Streams**
 - Lesen: InputStream bzw. Unterklasse
 - Schreiben: OutputStream bzw. Unterklasse
- **Textzeichen** (z. B. .txt, .java) → **Character-Streams**
 - Lesen: Reader bzw. Unterklasse
 - Schreiben: Writer bzw. Unterklasse

Beispiel: Datei-IO mit FileReader / FileWriter

```
// Öffnen einer Datei zum Lesen
FileReader fr = new FileReader(inFileName);

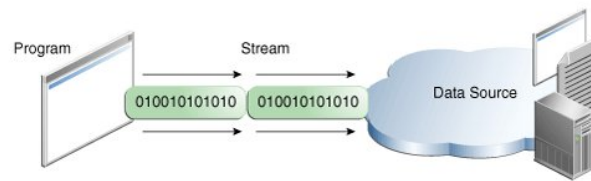
// Öffnen einer Datei zum Schreiben
FileWriter fw = new FileWriter(outFileName);

// Lesen eines Zeichens (Rückgabewert ist int)
int ch = fr.read();

// Schreiben des gelesenen Zeichens
fw.write(ch);
```

Datenfluss-Diagramm

Das folgende Diagramm visualisiert den Datenfluss zwischen einer Datenquelle (z. B. Netzwerk, Datei) und einem Java-Programm mittels Streams.



- **Links:** Das Programm empfängt einen **Input-Stream** (grüner Datenstrom).
- **Rechts:** Das Programm sendet Daten über einen **Output-Stream** zurück zur Quelle.

BufferedReader & BufferedWriter

- **BufferedReader**
 - Lies Zeichen effizient aus einem zugrundeliegenden Reader (z. B. FileReader).
 - Wichtigste Methode: `String readLine()` – liefert die nächste Zeile oder null am Dateiende.
 - Reduziert I/O-Aufrufe, weil mehrere Zeichen gleichzeitig im internen Puffer gespeichert werden.
- **BufferedWriter**
 - Schreibt Zeichen effizient in einen zugrundeliegenden Writer (z. B. FileWriter).
 - Wichtige Methoden: `write(String)`, `newLine()` (plattformunabhängiger Zeilenumbruch), `flush()` (Puffer leeren).
 - Verbessert die Schreibperformance, indem Daten gesammelt und in größeren Blöcken geschrieben werden.
- **Typisches Muster**

```
// Lesen
try (BufferedReader br = new BufferedReader(new FileReader("input.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        // Zeile verarbeiten
    }
}

// Schreiben
try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {
    bw.write("Erste Zeile");
    bw.newLine();           // Zeilenumbruch
    bw.write("Zweite Zeile");
    bw.flush();             // Puffer leeren (optional, wird bei close() automatisch)
}
```

- **Best Practices**
 - Immer **try-with-resources** verwenden, damit `close()` automatisch aufgerufen wird.
 - Für reine Byte-Daten `BufferedInputStream/BufferedOutputStream` einsetzen; für Textdaten immer `BufferedReader/BufferedWriter`.
 - Kombiniere `BufferedWriter` mit `PrintWriter`, wenn du formatierte Ausgaben (`printf`) brauchst.

Zusammenfassung der wichtigsten Punkte

Wrapper-Klassen ermöglichen die Nutzung primitiver Werte in objektbasierten Strukturen.

JAR-Dateien bündeln Klassen und Ressourcen für die Ausführung und Verteilung.

Streams abstrahieren den Zugriff auf interne und externe Datenquellen, wobei Byte- und Character-Streams je nach Datenart eingesetzt werden.